Read Deep, Write Shallow: How Looped Language Models Should Spend Their Depth

Yang Hu

Abstract

Looped language models reuse one block of layers several times per token and usually spend the same number of loops reading the prompt and writing the answer. We ask what each kind of depth buys. A small connector trained on a frozen looped model lets the prompt be read with p loops and the answer written with d loops, and one set of its parameters serves all 16 pairs up to four loops. On Ouro-2.6B the 4×4 map shows a division of labour: *depth spent reading the prompt mainly buys knowledge and long-context accuracy, while depth spent writing the answer mainly buys multi-step derivation*. With the writer fixed, reading deeper improves knowledge and long-context benchmarks on five checkpoints from two model families. Because writing is paid per token, writing shallow is what saves time: on Ouro-2.6B-Thinking a two-loop writer answers GSM8K in $0.55\times$ the time of the four-loop model, 1.0 point below it on the full test set. A RoPE-commuting key map lets one stored prompt KV cache serve every writing loop exactly. Linear probes find the next token more, and in Ouro the previous token less, linearly decodable from later loops' value cache. Code, results and per-item outputs: `https://github.com/yanghu819/read-deep-write-shallow`.

1 Introduction

Looped language models such as Ouro (Zhu et al., 2025), Huginn (Geiping et al., 2025) and Mixture-of-Recursions (Bae et al., 2025) obtain depth without adding parameters by running a shared block of layers several times per token (Dehghani et al., 2018). Under the fixed-depth inference we study, the price is paid twice: every generated token runs every loop, and every loop keeps its own key–value (KV) cache.

Yet the two phases of inference are not alike. Reading the prompt (prefill) is parallel and paid once per request; writing the answer (decode) is sequential and paid for every token. A looped model nevertheless spends the same depth on both. This raises a simple question: *what does depth buy when reading, and what does it buy when writing?* If the answers differ, a model could read deep and write shallow.

Answering the question needs a model that writes with fewer loops than it read with, so its shallow writing loops must attend to KV produced by a deeper loop. We introduce a small *depth-split connector*: LoRA on the frozen writer plus, for every writing loop, a map on the prompt KV (Figure 2). The key map commutes with rotary position embeddings, so one stored prompt cache serves every writing loop exactly. We train one connector by distillation from the native model and evaluate it at all 16 combinations of read and write depth up to four with its parameters fixed, which lets us measure the two effects separately.

The resulting map shows a division of labour (Figure 1). On the benchmarks we test, reading depth mainly moves knowledge and long-context accuracy, and writing depth mainly moves multi-step derivation. With the writer held fixed, reading deeper helps on five checkpoints (Table 2), although the effect depends on the task. Because writing is paid per token, shallow writing is where time is saved: a two-loop writer answers in about half the time of the four-loop reasoning model, one point below it on the full GSM8K test set (Figure 3). Reading deeper also helps a one-loop writer finish its answer; on SenseNova-Loop-9B this holds even without any training (Figure 4). Linear probes of the cache are consistent with the split: later loops make the next token more linearly decodable and, in Ouro, the previous token less (Figure 5); correspondingly, reading deep is worse for short-context retrieval and few-shot mathematics.

Our contributions are a measurement of what

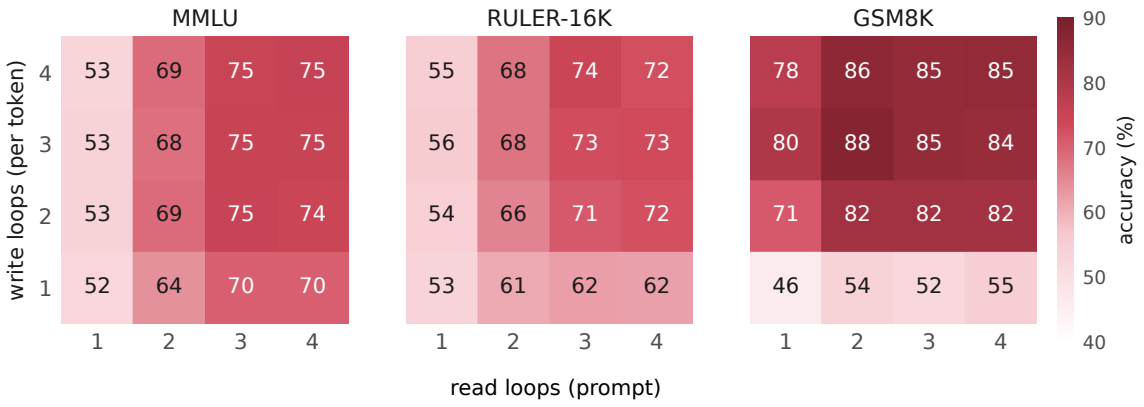


Figure 1: **Reading depth and writing depth buy different things.** One connector on frozen Ouro-2.6B evaluated at all 16 (read, write) pairs. Knowledge (MMLU) and long-context reading (RULER-16K) improve along the read axis; multi-step derivation (GSM8K) improves along the write axis. Grid subsets: MMLU 20 questions per subject, RULER-16K 260 rows, GSM8K 400 problems; one colour scale for all panels; full-set comparisons are in Table 1.

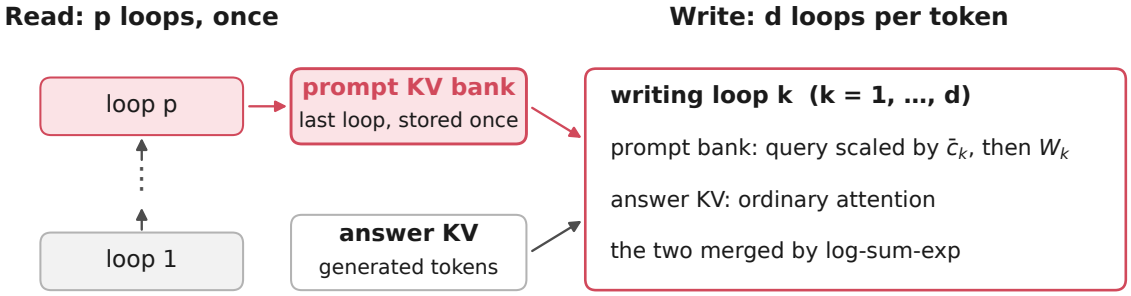


Figure 2: **The depth-split connector.** The frozen model reads the prompt with p loops and keeps one prompt KV bank. Each of the d writing loops reads that bank through its own map: a per-frequency complex key scale, folded into the query because it commutes with RoPE, and a value map applied after attention. The prompt part is merged by log-sum-exp with ordinary attention over the generated tokens' KV. The prompt KV is stored once for every d: total KV $b(n + d\,t)$ instead of $b\,d(n + t)$, with b bytes per token and loop and n, t the prompt and answer length.

reading and writing depth each buy in looped language models, a connector that makes the split usable on existing checkpoints with an exact single prompt cache, and an analysis of what each loop's KV stores. Code, per-item results and a script that recomputes every number in the paper accompany it.

2 Related Work

Looped and elastic depth. Accordion (Zhang et al., 2026) and Elastic Latent Recurrence (Ladhak et al., 2026) train one model to serve several depth budgets; ELR also reports that keeping only the last loop's KV at inference retains most of the gain. The Parallel Loop Transformer (Wu et al., 2025a), LoopCoder-v2 (Yang et al., 2026) and shared-memory looped transformers (Monea et al., 2026) let later loops read the first loop's global KV plus a short window of their own. All of them spend the same depth on prompt and answer.

Phase-specific computation. Continuous Depth Batching (Schwethelm et al., 2026) runs full-depth prefill and exits early during decode; Position-Aware Depth Decay (Fan et al., 2025) skips more layers for later tokens; OverFill (Kim et al., 2025) prefills with the full model and decodes with a width-pruned one. Fixed Points Part II (Huang et al., 2026) distils a lightweight prefill network, followed by one teacher recurrence, while keeping the teacher's decoding depth; POP (He et al., 2026) and SPEED (Oh et al., 2026) use shallower prefill than decode. Decode-Branch Transformers (Liu et al., 2026), PHD (Wu et al., 2025b) and Almost Free State-Prediction Separation (Langford et al., 2026) add decode-side computation. These works change decode depth, reduce prefill computation or share caches across recurrences; we instead vary prompt and answer depth independently within one trained connector

and measure what each buys.

Sharing and transferring KV. Looped Latent Attention (Neill and Reid, 2026) compresses KV across loops; BeNice! (Prairie et al., 2026) and LSTM-style looped transformers (Kirsch, 2026) share it; Cache-to-Cache (Fu et al., 2025), HeteroFold (Yun et al., 2026), Swarm Telepathy (Alabdulkareem et al., 2026) and CacheLink (Qiu et al., 2026) transfer KV between models; BellowsKV (Zhang and Chen, 2026) shares one latent across layers. As in CacheLink's output-supervised corrections, our maps are trained through the receiver's output distribution. Within one Ouro model, tokens, layers and positions align, so a RoPE-commuting scale and a value map suffice, and serving them from one stored cache is exact.

3 Method

We describe the connector for Ouro; SenseNova-Loop-9B uses a lighter variant without maps (Appendix A).

Reading and writing. "read p / write d" prefills the prompt with p loops and decodes every answer token with d loops; native d/d is the unmodified model. The frozen model reads the prompt with p loops and keeps only the last loop's prompt keys and values. The writer is the same frozen model with rank-16 LoRA (Hu et al., 2021) on its attention projections, running d loops per token.

RoPE-commuting maps. Writing loop k reads the prompt bank through a map per (loop, layer, KV head). The key map is one complex scale c_j per rotary frequency pair j (Su et al., 2021). Writing a rotated key pair as a complex number $\tilde{k}_j = e^{im\theta_j} k_j$, scaling commutes with the rotation, $c_j \tilde{k}_j = e^{im\theta_j}(c_j k_j)$, and $\mathrm{Re}\langle q_j, c_j \tilde{k}_j \rangle = \mathrm{Re}\langle \bar{c}_j q_j, \tilde{k}_j \rangle$: the map is applied to the query and the stored cache is never rewritten. The value map is a full linear map W_k per head; because attention is linear in the values, $\sum_i a_i W_k v_i = W_k \sum_i a_i v_i$ is applied once to the attention output over the prompt. Prompt and generated tokens share one softmax: the two parts are computed by two attention calls and merged by their log-sum-exp. The prompt KV is therefore stored once for every d ("single bank"), and total KV is $b(n + dt)$ instead of $b\,d(n + t)$. The single-bank form equals materialising a mapped prompt bank per loop; in bfloat16 all 512 checked greedy tokens agree.

Storing the answer once. Generated tokens can also share one KV bank across writing loops, which reduces total KV to $b(n + t)$. This optional variant changes training and costs accuracy at four writing loops (Appendix A, Table 6).

Training. We minimise $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ over answer positions, with the native 4/4 model as teacher, on 48M answer tokens of reasoning traces, instruction data, long-context data and web text. One (p, d) cell is sampled per micro-batch, so one set of parameters (63M, 2.4% of Ouro-2.6B) serves all 16 cells. Details are in Appendix A.

4 Experiments

Setup. Models: Ouro-2.6B, Ouro-1.4B and their Thinking variants (Zhu et al., 2025) (four loops), Nanbeige4.2-3B-Base (two loops) and SenseNova-Loop-9B-Preview, a hybrid of Gated-DeltaNet and full-attention layers whose middle block is visited twice. Benchmarks: GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021; Lightman et al., 2023), MMLU (Hendrycks et al., 2020), MMLU-Pro (Wang et al., 2024), ARC-Challenge (Clark et al., 2018), BBH (Suzgun et al., 2022), RULER (Hsieh et al., 2024) at 4K/8K/16K and HELMET (Yen et al., 2024); protocols and set sizes are in Appendix B. Every comparison is paired over items with 95% bootstrap intervals. To isolate reading depth from adaptation we compare read p / write d with read d / write d of the *same* connector; comparisons with native models include the effect of training the connector.

4.1 Reading buys understanding, writing buys derivation

Figure 1 shows the full grid on Ouro-2.6B. MMLU rises from 52–53 at one read loop to 70 at three or four with a one-loop writer and to 74–75 with deeper writers; GSM8K barely moves along a row at one write loop and rises from 46–55 to 78–88 with writing depth. Table 1 confirms the pattern on full sets: at one writing loop, reading four loops instead of one adds 15.2 MMLU, 8.6 RULER-16K and 8.0 GSM8K points; the gain shrinks as writing deepens. A second training seed reproduces these numbers (Appendix C).

The same holds across models (Table 2). With the writer fixed at one loop, reading deeper improves knowledge and long-context tasks and the selected multi-step task on all five check-

write d	MMLU	RULER-16K	GSM8K
1	$+15.2$ [14.3, 16.1]	$+8.6$ [4.3, 12.8]	$+8.0$ [5.3, 10.5]
2	$+6.0$ [5.4, 6.7]	$+5.3$ [1.3, 9.4]	-1.7 [-3.6, 0.2]
3	$+0.6$ [0.2, 1.1]	-0.4 [-3.0, 2.2]	-0.8 [-2.4, 0.8]

Table 1: **Reading depth at fixed writing depth** (Ouro-2.6B, one connector, full sets): read 4 / write d minus read d / write d.

points; Ouro-1.4B is the exception on long context (RULER-16K $+1.4$, not significant). The gains are task-dependent: on both 1.4B checkpoints the same change sharply lowers 3-shot BBH (Limitations). On Nanbeige4.2-3B-Base separately trained connectors per cell point the same way but do not isolate reading depth (Appendix B).

4.2 Writing shallow is fast

Section 4.1 asked whether reading deeper helps a fixed writer. Here we ask whether a shallower trained writer saves time, measuring whole answers (Figure 3, Table 3). On Ouro-2.6B-Thinking a connector that writes with two loops answers GSM8K in $0.55\times$ [0.45, 0.67] the time of native 4/4. On all 1,319 problems, read 4 / write 2 scores 93.3 against 94.2, a paired difference of -1.0 [-2.0, 0.1]. Reading 4 rather than 2 loops before writing two takes $1.01\times$ [0.90, 1.11] the time: deeper reading costs no detectable time, and the saving against the native model comes from the shallower, adapted writer, including its shorter answers. On the 40 timed MATH-500 problems read 4 / write 2 scores 85.0, against 82.5 for native 4/4 and 85.0 for native 2/2, at $0.63\times$ the time of native 4/4 but more than native 2/2, because its solutions are not shorter. On SenseNova-Loop-9B the trained read 1 / write 1 scores 95.6 against 95.8 for native 2/2 on all 1,319 GSM8K problems and takes $0.59\times$ [0.52, 0.68] the time per answer, close to the ratio of layer executions per token (32 vs. 56).

Serving. Storing the prompt once also changes throughput (Table 4). In a controlled decode benchmark (8K context, 32 repeated decode steps, largest batch that fits on one B300), the single bank decodes $0.97\times$ as fast as the fastest native implementation we measured at one writing loop and 1.19–$1.62\times$ at two to four, because it admits larger batches as d grows; this native baseline uses the same flash attention and is faster than the standard dynamic cache (Appendix B). With long answers (1K-token prompts, 2,048 generated tokens) the optional shared answer bank more than doubles write-4 throughput (219 to 476 tokens/s), at a cost of 4.8 GSM8K points against native 4/4.

4.3 Deep reading helps a shallow writer

A reasoning model written with one loop often does not finish. On Ouro-2.6B-Thinking, native 1/1 gives no boxed answer on 43.1% of GSM8K problems; 43.5% of its answers run into the 4,096-token budget. Training the connector at read 1 / write 1 reduces answers without a boxed answer to 19.5%; with the same writer, reading four loops reduces them further to 12.7% in every difficulty bucket (Figure 4, left) and shortens answers by 20% overall. A lexical hesitation-marker rate does not fall (12.2 vs. 13.8 per 1,000 tokens), so shorter answers do not come from fewer markers of hesitation; these counts do not identify the underlying reasoning process. SenseNova-Loop-9B gives a test without training, because its one-loop writer collapses. Handing that writer the two-visit prompt state—attention KV and Gated-DeltaNet state, no parameter changed—cuts answers without a boxed answer from 96.8% to 43.7% and lifts RULER-4K from 21.6 to 83.5 and MMLU-Pro from 18.9 to 55.1 (Figure 4, right); GSM8K accuracy, however, only rises from 2.2 to 10.9, far below the 95.8 of native 2/2.

4.4 What each loop stores

What changes in the cache from loop to loop? We fit linear probes to the frozen native model's per-loop value cache—what attention reads back from the prompt—at the same layer and with the same weights, varying only the loop (Figure 5). From loop 1 to loop 4 of Ouro-2.6B the previous token becomes less linearly decodable (57.7% $\rightarrow$ 45.5%, lower in 12 of 13 probed layers) and the next token more (26.0% $\rightarrow$ 36.1%, higher in all 13); Ouro-1.4B and Ouro-2.6B-Thinking agree. In SenseNova-Loop-9B the next-token gain replicates in all six attention layers (24.1% $\rightarrow$ 29.1%) while the previous token is about as decodable at both visits (56.8% vs. 56.5%). These are decodability results, not proof of information loss or causal use (Merin, 2026), and the probes read the native model, not the adapted writer; they resemble the across-layer trend of Voita et al. (2019), and per-loop probes of looped transformers go back to Yang et al. (2023).

Reading deeper does not help everywhere (Table 5). At one writing loop it gives a negative

model (same writer, one writing loop)	knowledge	long context	multi-step
Ouro-2.6B, read 4 vs. 1	MMLU +15.2 [14.3, 16.1]	RULER-16K +8.6 [4.3, 12.8]	GSM8K +8.0 [5.3, 10.5]
Ouro-1.4B, read 4 vs. 1	ARC-C +10.8 [8.4, 13.1]	RULER-16K +1.4 [-2.6, 5.4]	GSM8K +6.2 [3.6, 8.7]
Ouro-2.6B-Thinking, read 4 vs. 1	MMLU-Pro +18.6 [13.6, 23.8]	HELMET +12.0 [9.1, 15.0]	GSM8K +6.6 [4.2, 9.2]
Ouro-1.4B-Thinking, read 4 vs. 1	MMLU-Pro +17.1 [12.4, 22.4]	HELMET +11.9 [9.0, 14.8]	GSM8K +13.6 [10.5, 16.6]
SenseNova-Loop-9B, read 2 vs. 1	MMLU-Pro +5.5 [2.9, 8.1]	HELMET +6.7 [5.7, 7.8]	BBH +7.0 [5.9, 8.0]

Table 2: **Reading depth with the writer held fixed** across five checkpoints (paired, within one connector). Set sizes: MMLU 14,042, ARC-C 1,172, MMLU-Pro 420 (Ouro) and 980 (SenseNova), RULER 520, HELMET 1,108 (Ouro) and 6,646 (SenseNova), GSM8K 1,319, BBH 6,511.

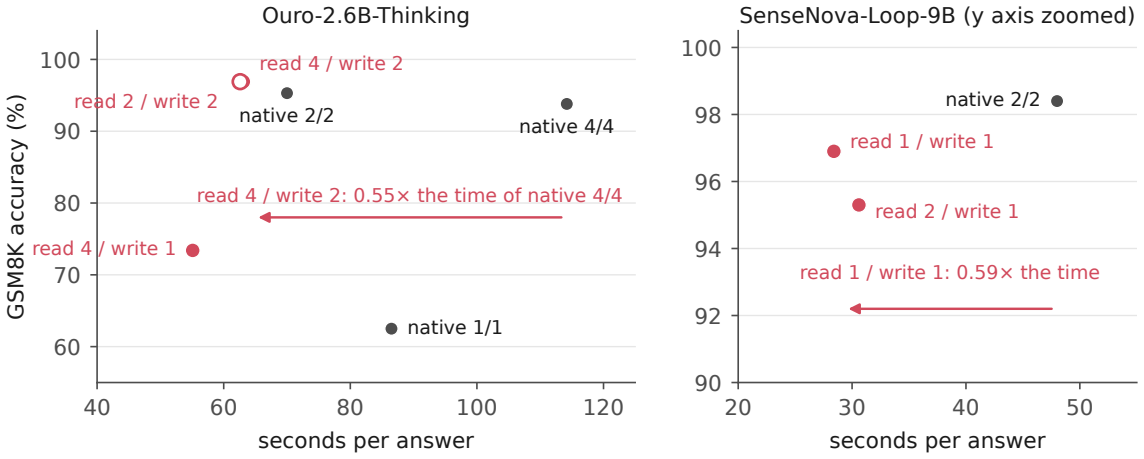


Figure 3: **Writing shallow saves time end to end.** Accuracy vs. seconds per answer at batch 1 (prefill included, real greedy answers, the same 64 GSM8K problems for every point). Left: writing with two loops takes 0.55× the time of native 4/4, whether the prompt is read with 2 or 4 loops; on all 1,319 problems read 4 / write 2 is 1.0 point below native 4/4 (Section 4.2). Right: on SenseNova-Loop-9B, whose native one-loop model solves 2.2% of all GSM8K problems, read 1 / write 1 is 1.5 points below native 2/2 at 0.59× the time (y axis zoomed).

	config	acc.	s / answer
GSM8K Ouro-2.6B-T	native 2/2	95.3	70.0
	native 4/4	93.8	114.2
	read 2 / write 2	96.9	62.6
	read 4 / write 2	96.9	63.1
MATH-500 Ouro-2.6B-T	native 2/2	85.0	243
	native 4/4	82.5	469
	read 4 / write 2	85.0	298
GSM8K SenseNova	native 2/2	98.4	48.0
	read 1 / write 1	96.9	28.4
	read 2 / write 1	95.3	30.6

Table 3: **End to end per answer** (batch 1, one B300; 64 GSM8K and 40 MATH-500 problems, budgets 4,096 and 8,192 tokens). On GSM8K, read 4 / write 2 costs 1.27× native 2/2 per generated token at batch 1, but its answers are shorter.

8K context, 32 steps	d=1	d=2	d=3	d=4
native d/d	834	342	200	125
read 4 / write d, one prompt bank	813	406	271	203
1K context, 2,048 steps		d=2		d=4
native d/d		586		219
one prompt bank		570		253
prompt and answer once		759		476

Table 4: **Decode throughput** (tokens/s, best over tested batch sizes, one B300). Largest tested batch that fits, long answers at d=4: 32 / 64 / 192.

but inconclusive estimate on RULER-4K, where answers are copied verbatim, and clearly lowers 4-shot MATH-500 on Ouro-2.6B; without exemplars the same comparison turns positive on two model sizes and two writing depths. This fits the probes, but removing exemplars also changes length and format, so we read it as a prompting interaction rather than as evidence that literal information is lost.

4.5 Design variants

Table 6 compares design variants. A per-cell, LoRA-only writer that reads the last prefill loop's KV without a map scores 17.5 GSM8K points below our connector at read 4 / write 1 and ties it at write 2; this terminal-KV baseline is inspired by cache sharing (Prairie et al., 2026) but does not

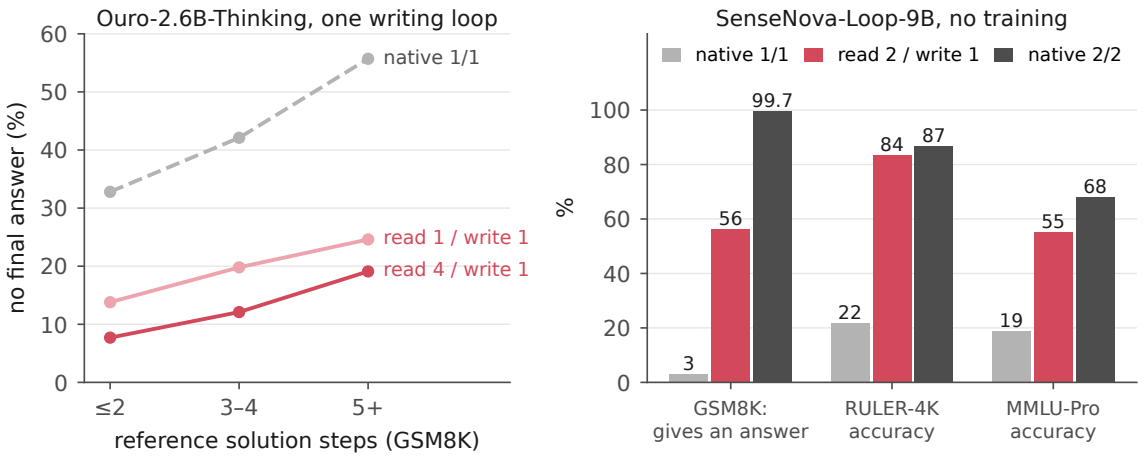


Figure 4: **Deep reading helps a shallow writer.** Left: Ouro-2.6B-Thinking at one writing loop, GSM8K problems bucketed by the number of lines in the reference solution (326 / 668 / 325 problems). Answers without a boxed answer grow with difficulty; reading four loops instead of one lowers them by 5.5–7.7 points in every bucket. Right: SenseNova-Loop-9B without any training: the native one-loop writer reads the prompt state of a two-visit prefill.

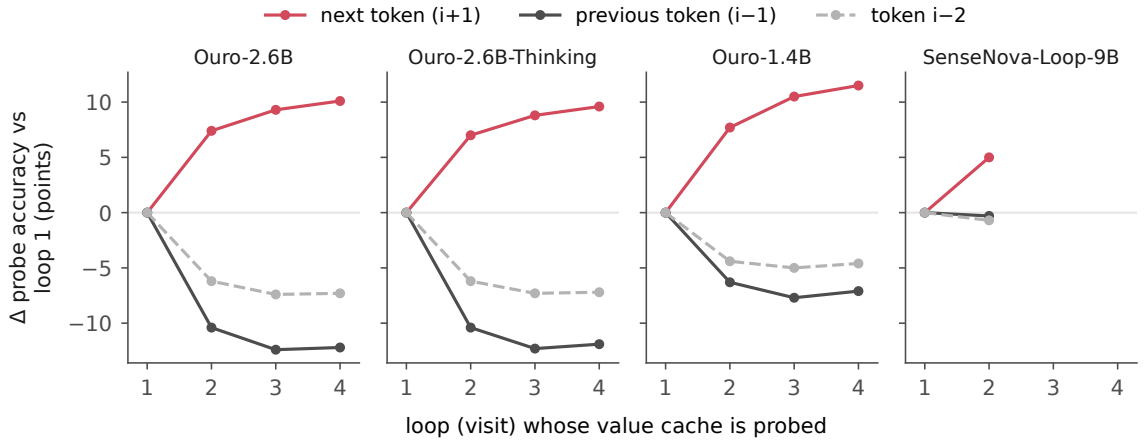


Figure 5: **Later loops make the next token more decodable; in Ouro the previous token becomes less decodable.** Linear probes on the stored value cache, same weights and layer, change from loop 1 (mean over probed layers; 384 FineWeb windows of 1,024 tokens, 20% held out; 2,048 most frequent training tokens).

model	write d	task	read 4 − read d
Ouro-2.6B	1	RULER-4K	−3.8 [-8.2, 0.3]
	1	MATH, 4-shot	−11.2 [-15.4, -6.8]
	1	MATH, 0-shot	+7.0 [3.0, 10.8]
	2	MATH, 0-shot	+7.0 [3.0, 11.2]
Ouro-1.4B	1	MATH, 4-shot	−4.0 [-8.2, 0.0]
	1	MATH, 0-shot	+4.8 [1.4, 8.4]

Table 5: **Reading deep depends on context length and prompting.** Same connector, base models, read 4 / write d minus read d / write d; MATH-500 scored with math-verify on the full answer.

reproduce its recipe, and it also differs from our connector in training allocation and tokens, so it does not isolate the map. Restricting the key map to a RoPE-commuting scale, which is what allows one stored prompt bank, costs nothing measurable against unrestricted key maps with one bank per loop. Storing the answer once shows no significant difference at two and three writing loops and costs 2.6 GSM8K points at four. A gated branch over the first prefill loop's KV improves RULER-4K by 2.3 points at read 4 / write 1, leaves GSM8K unchanged within noise, and does not remove the 4-shot MATH deficit.

5 Conclusion

Depth in a looped language model is two budgets, not one. On the checkpoints and benchmarks we test, reading depth is paid once and mainly buys knowledge and long-context accuracy, while writing depth is paid per token and mainly buys multi-

variant	p/d	task	difference
direct sharing†	4/1	GSM8K	-17.5 [-20.1, -14.9]
(no map)	4/2	GSM8K	-0.1 [-2.0, 1.7]
unrestricted keys	5 cells	RULER-16K	all intervals $\ni 0$
answer once	4/2	GSM8K	$+0.3$ [-1.5, 2.1]
	3/3	GSM8K	-0.8 [-2.4, 0.8]
	4/4	GSM8K	-2.6 [-4.2, -1.1]
first-loop branch	4/1	RULER-4K	$+2.3$ [1.0, 3.6]
	4/1	GSM8K	$+0.7$ [-0.8, 2.2]

Table 6: **Design variants** on Ouro-2.6B: variant minus our connector at read p / write d (full sets, paired). †LoRA only, one connector per cell, 24M KL tokens; the other variants follow our recipe.

step derivation; both effects depend on the task. A small connector makes the two separable on existing checkpoints with one exact prompt cache, and a shallower adapted writer cuts the measured time per answer at a small accuracy cost.

Limitations

Reading deep is worse for short-context retrieval and 4-shot mathematics (Table 5); at three or more writing loops, one deep prompt bank trails the native model on RULER ($-10.4/-9.5/-5.3$ at 4K/8K/16K, write 3). Deeper prefill raises time to first token, so it pays when answers are long or quality requires it. Both 1.4B checkpoints degenerate at read 4 / write 1 on 3-shot BBH (Ouro-1.4B: -24.5, Ouro-1.4B-Thinking: -20.5 against read 1 / write 1); with two writing loops, reading four instead of two helps (Ouro-1.4B-Thinking: $+8.9$). On Nanbeige4.2-3B-Base our universal recipes trail per-cell connectors, so Appendix B reports per-cell results. Gains against native models include the adaptation of the connector. Throughput is a controlled decode benchmark and end-to-end time is measured at batch 1 on one GPU type. Thinking checkpoints on raw HELMET prompts can stop at once; we decode them so that an answer cannot be empty, for every configuration (Appendix B). Probes measure linear decodability, not causal use. All models are 1.4B–9B, decoding is greedy and benchmarks are English.

References

Abdulrahman Alabdulkareem, Irene Tenison, Yang Zhang, Yada Zhu, Boris Katz, and Dan Gutfreund. 2026. Swarm Telepathy: Prefill-Free KV-Cache Transfer Across Families. *arXiv preprint arXiv:2610.15095*.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2018. Universal Transformers. *arXiv preprint arXiv:1807.03819*.

Siqi Fan, Xuezhi Fang, Xingrun Xing, Peng Han, Shuo Shang, and Yequan Wang. 2025. Position-Aware Depth Decay Decoding (D^3): Boosting Large Language Model Inference Efficiency. *arXiv preprint arXiv:2503.08524*.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2025. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. *arXiv preprint arXiv:2510.03215*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*.

Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. 2026. POP: Prefill-Only Pruning for Efficient Large Model Inference. *arXiv preprint arXiv:2602.03295*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring Massive Multitask Language Understanding. *arXiv preprint arXiv:2009.03300*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv preprint arXiv:2103.03874*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. 2026. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*.

Woojeong Kim, Junxiong Wang, Jing Nathan Yan, Mohamed Abdelfattah, and Alexander M. Rush. 2025. OverFill: Two-Stage Models for Efficient Language Model Decoding. *arXiv preprint arXiv:2508.08446*.

Andreas Kirsch. 2026. LSTM-Style Looped Transformers with Shared KV Caches. *arXiv preprint arXiv:2610.33360*.

Faisal Ladhak, Shantanu Acharya, Santiago Akle Serano, Simeng Sun, Boris Ginsburg, and Krishna C Puvvada. 2026. One Model to Loop Them All: Elastic Latent Recurrence for Language Models. *arXiv preprint arXiv:2610.12582*.

John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. 2026. Almost Free State Prediction Separation. *arXiv preprint arXiv:2609.03807*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050*.

Liming Liu, Mingze Wang, and Tuo Zhao. 2026. Decode-Branch Transformers: Decoupling the Primary Prefill Path from Additional Decode Computation. *arXiv preprint arXiv:2608.12385*.

Aur Shalev Merin. 2026. Present but Hard to Read: Probing Latent Reasoning with Minimal Pairs. *arXiv preprint arXiv:2610.28498*.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. 2026. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*.

James O' Neill and Fergal Reid. 2026. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. *arXiv preprint arXiv:2607.15456*.

Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow Prefill, Deep Decoding: Efficient Long-Context Inference via Layer-Asymmetric KV Visibility. *arXiv preprint arXiv:2605.06105*.

Hayden Prairie, Jacob M. Ryan, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. 2026. BeNice! Closing Looping's Inference-Memory Gap via KV-Sharing. *arXiv preprint arXiv:2610.31633*.

Haomiao Qiu, Yining Tang, Zijia Song, Ruizi Han, Ziyue Qiao, and Miao Zhang. 2026. CacheLink: Functional KV Transport for Zero-Prefill Cross-Model Context Handoff. *arXiv preprint arXiv:2610.15509*.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. 2026. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. *arXiv preprint arXiv:2608.09444*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv preprint arXiv:2210.09261*.

Elena Voita, Rico Sennrich, and Ivan Titov. 2019. The Bottom-up Evolution of Representations in the Transformer: A Study with Machine Translation and Language Modeling Objectives. In *Proceedings of EMNLP-IJCNLP 2019*, pages 4396–4406.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv preprint arXiv:2406.01574*.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. 2025a. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. 2025b. Efficient Pretraining Length Scaling. *arXiv preprint arXiv:2504.14992*.

Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. 2026. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2606.18023*.

Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. 2023. Looped Transformers are Better at Learning Learning Algorithms. *arXiv preprint arXiv:2311.12424*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat,

and Danqi Chen. 2024. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. *arXiv preprint arXiv:2410.02694*.

Vincent-Daniel Yun, Woosang Lim, Haneul Yoo, Sungjoo Yoo, Murali Annavaram, and Sai Praneeth Karimireddy. 2026. Prefill-Free Cross-Family KV Cache Transfer for Heterogeneous Multi-Agent LLMs. *arXiv preprint arXiv:2609.32259*.

Jiamu Zhang and Hanjie Chen. 2026. BellowsKV: One Latent for Many Layers, Expanded on Demand. *arXiv preprint arXiv:2610.35023*.

Yudong Zhang, Wei Ding, Kaili Zheng, Yuchen Ding, and Zhenyu Zhang. 2026. Accordion: Elastic-Depth Inference via Fixed-Prefix Layer Looping. *arXiv preprint arXiv:2610.32492*.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

A Training Details

Connector. Rank-16 LoRA ($\alpha = 16$, no dropout) on the query, key, value and output projections, active only when writing; per (writing loop, layer, KV head) a complex scale for each rotary frequency pair on keys and a full $D \times D$ value map, both initialised to the identity. Trainable parameters: 63,307,776 for Ouro-2.6B. The backbone is frozen.

Objective and data. $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ summed over answer positions and normalised by the KL tokens in the global batch; teacher = native 4/4 with the same weights; one cell (p, d) sampled per micro-batch. 48M KL tokens: OpenThoughts reasoning traces 45%, Tulu SFT responses 40%, LongAlpaca long-context answers 5% and FineWeb-Edu windows 10% (fractions of KL tokens); prompts up to 7,680 tokens, answers up to 1,024. Base models see a plain "User: … Assistant:" format; thinking models see the same samples in their chat template. No benchmark data was deliberately included; we did not decontaminate the training corpora.

Optimisation. AdamW ($\beta = (0.9, 0.95)$, no weight decay), peak learning rate 10^{-4} with 2% linear warm-up and cosine decay to 10^{-5}, gradient clipping at 1.0, global batch 24 sequences on 4 B300 GPUs (6.75 hours for Ouro-2.6B), bfloat16 backbone.

Answer stored once. Generated tokens keep one KV bank per layer shared by all writing loops; at inference the last writing loop commits each token, and earlier loops read that history plus the current token's KV computed live in their own loop (*self-live*[1], a layout also used by LSTM-style looped transformers (Kirsch, 2026)). Training uses the same KL objective in this layout, in two passes: a no-grad pass in which each loop reads its own causal answer KV records the last loop's answer KV; a second pass reads, at every loop, the prompt bank, that recorded history strictly before the current position, and the current token's KV computed live on the diagonal. Training thus approximates, but does not exactly reproduce, inference, where the history is generated recursively in the shared bank.

SenseNova-Loop-9B. Rank-16 LoRA on the attention projections of the looped layers (full attention: q/k/v/o; Gated-DeltaNet: input and output projections), active only when writing; no maps. The one-visit writer reads the prefill state of the last of $p \in \{1, 2\}$ visits (sampled 50/50), so the same writer serves read 1 and read 2. The untrained condition passes the last visit's full-attention KV and Gated-DeltaNet convolution and recurrent states. Teacher: native 2/2; 20M KL tokens of the same mixture in the SenseNova chat template; peak learning rate 2×10^{-4}.

B Evaluation Details

Base models use lm-evaluation-harness prompts through our runner for GSM8K (3-shot CoT, flexible-extract, 1,319), MMLU (5-shot, 14,042), ARC-Challenge (25-shot, 1,172), BBH (3-shot CoT, 60 per subtask) and MATH-500 (the 500-problem subset of MATH, 4-shot Minerva prompt). RULER (13 tasks $\times$ 40 rows per length) and HELMET (RAG, synthetic recall, re-ranking and ICL categories; 1,108 rows) are prepared and scored by our own scripts. Thinking models answer GSM8K and MATH-500 zero-shot in their chat template, answers in `\boxed{}` scored with math-verify, with budgets of 4,096 and 8,192 tokens; their MMLU-Pro is 5-shot CoT with 30 questions per subject (420) and their BBH 3-shot CoT with 60 per subtask. SenseNova-Loop-9B uses MMLU-Pro with 70 questions per subject

[1] Self-live was developed by the author in separate work on shared KV caches for looped models; here it is applied to the connector's answer tokens.

(980) through the harness and the full BBH (6,511) through our own scripts. For base models, MATH-500 answers are scored with math-verify on the full generated answer, with and without exemplars. The harness's own math-verify extraction gives -9.8 instead of -11.2 for the 4-shot comparison on Ouro-2.6B; Minerva's strict "Final Answer" match would reverse it (19.6 vs. 9.8) because the read-1 writer drops the required format more often. On raw HELMET prompts thinking checkpoints can emit a newline and stop (42.7% of native 2/2 answers on Ouro-2.6B-Thinking); for thinking models we mask the stop tokens until an answer has a non-whitespace token, for every configuration; this does not rescue native 2/2 (34.1 vs. 31.4). SenseNova RULER uses a non-thinking prefill for every configuration. Native throughput baselines: static per-loop banks with the same two-call flash attention (fastest), the standard dynamic cache (500 / 245 / 105 tokens/s at $d = 1/2/4$), and a preallocated contiguous cache with one attention call (267 / 130 / 57); all produce identical greedy tokens. Nanbeige4.2-3B-Base per-cell connectors, read 2 / write 1 minus read 1 / write 1: RULER 4K/8K/16K $+11.8/+11.7/+14.2$, MMLU $+12.2$, GSM8K $+5.2$. Intervals: paired bootstrap over items, 5,000 resamples; every interval in the paper is recomputed from released per-item scores by one script (Appendix D).

C Training-Seed Robustness

	task	seed 1	seed 2
read 4 − read 1 (write 1)	GSM8K	$+8.0$	$+7.2$
	MMLU	$+15.2$	$+15.2$
	RULER-4K	-3.8	-3.5
	RULER-8K	$+2.5$	$+2.0$
	RULER-16K	$+8.6$	$+8.6$
read 4 − read 2 (write 2)	GSM8K	-1.7	-2.9
	RULER-4K	-7.2	-6.8

Table 7: Ouro-2.6B connector trained with two seeds (initialisation and cell sampling); paired differences within each seed, full sets.

D Reproducibility

`python ouro/scripts/paper_numbers.py compute` recomputes every paired difference, interval, mean, timing ratio and Section 4.3 statistic in this paper from the accompanying per-item records (1 MB) in seconds on a laptop, and checks them against the printed values. `REPRODUCE.md` maps the remaining results (throughput, probes) to their scripts and gives the training and evaluation commands that regenerate the per-item records.